

PLAN DE PRUEBAS – ECOCHARGER

1. Introducción

Este documento recoge el plan de pruebas realizado sobre la aplicación **EcoCharger**, incluyendo pruebas de caja negra y caja blanca tanto en la capa de **Services** como en la capa de **ViewModels**.

El objetivo es verificar el correcto funcionamiento del sistema, asegurando que cada operación cumple con el comportamiento esperado.

2. Alcance de las pruebas

Las pruebas realizadas cubren:

- Operaciones CRUD de los Services
- Comandos de los ViewModels
- Validaciones internas
- Gestión de datos en base de datos SQL Server (LocalDB)

3. Tipos de pruebas realizadas

3.1 Pruebas de caja negra

Se comprueba el comportamiento del sistema a partir de entradas y salidas esperadas, sin tener en cuenta la implementación interna.

3.2 Pruebas de caja blanca

Se prueban condiciones internas del código, como validaciones, excepciones y ramas condicionales.

4. Pruebas realizadas sobre Services

SERVICE CHARGER

◆ Prueba 1 – Listar cargadores

Objetivo:

Comprobar que el sistema devuelve correctamente la lista de cargadores almacenados en la base de datos.

Datos de entrada:

Sin parámetros.

Datos esperados:

Lista no nula de cargadores.

Datos obtenidos:

Lista correctamente devuelta.

Resultado: ✓ Correcto

◆ Prueba 2 – Insertar cargador válido

Objetivo:

Verificar que se inserta correctamente un nuevo cargador asociado a una estación existente.

Datos de entrada:

Charger válido:

- Type = 1
- MaxPower = 50
- IsOccupied = false
- StationId válido

Datos esperados:

Cargador insertado con Id > 0.

Datos obtenidos:

Id generado correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Insertar cargador null

Objetivo:

Comprobar que el sistema controla correctamente valores nulos.

Datos de entrada:

null

Datos esperados:

Excepción ArgumentNullException.

Datos obtenidos:

Excepción lanzada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Actualizar cargador existente

Objetivo:

Verificar que un cargador existente puede ser modificado.

Datos de entrada:

Cargador previamente insertado con modificación en MaxPower.

Datos esperados:

Resultado true.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Actualizar cargador inexistente

Objetivo:

Comprobar que el sistema no actualiza registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar cargador existente

Objetivo:

Comprobar que un cargador puede eliminarse correctamente.

Datos de entrada:

Id válido previamente insertado.

Datos esperados:

Resultado true.

Datos obtenidos:

Eliminación correcta.

Resultado: ✓ Correcto

◆ Prueba 7 – Borrar cargador inexistente

Objetivo:

Verificar que el sistema controla correctamente la eliminación de registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

SERVICE STATION

◆ Prueba 1 – Listar estaciones

Objetivo:

Comprobar que el sistema devuelve correctamente la lista de estaciones almacenadas en la

base de datos junto con sus cargadores asociados.

Datos de entrada:

Sin parámetros.

Datos esperados:

Lista no nula de estaciones.

Datos obtenidos:

Lista correctamente devuelta.

Resultado: ✓ Correcto

◆ Prueba 2 – Insertar estación válida

Objetivo:

Verificar que se inserta correctamente una nueva estación en la base de datos.

Datos de entrada:

Station válida:

- Name = "Station_Test"
- Address = "Dirección Test"
- Latitude = 0
- Longitude = 0
- IsActive = true

Datos esperados:

Estación insertada con Id > 0.

Datos obtenidos:

Id generado correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Insertar estación null

Objetivo:

Comprobar que el sistema controla correctamente la inserción de valores nulos.

Datos de entrada:

null

Datos esperados:

Excepción ArgumentNullException.

Datos obtenidos:

Excepción lanzada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Actualizar estación existente

Objetivo:

Verificar que una estación existente puede ser modificada correctamente.

Datos de entrada:

Estación previamente insertada con modificación en Name.

Datos esperados:

Resultado true.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Actualizar estación inexistente

Objetivo:

Comprobar que el sistema no actualiza registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar estación existente

Objetivo:

Comprobar que una estación puede eliminarse correctamente.

Datos de entrada:

Id válido previamente insertado.

Datos esperados:

Resultado true.

Datos obtenidos:

Eliminación correcta.

Resultado: ✓ Correcto

◆ Prueba 7 – Borrar estación inexistente

Objetivo:

Verificar que el sistema controla correctamente la eliminación de registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

SERVICE TARIFF

◆ Prueba 1 – Listar tarifas

Objetivo:

Comprobar que el sistema devuelve correctamente la lista de tarifas almacenadas en la base de datos.

Datos de entrada:

Sin parámetros.

Datos esperados:

Lista no nula de tarifas.

Datos obtenidos:

Lista correctamente devuelta.

Resultado: ✓ Correcto

◆ Prueba 2 – Insertar tarifa válida

Objetivo:

Verificar que se inserta correctamente una nueva tarifa en la base de datos.

Datos de entrada:

Tariff válida:

- Name = "Tarifa_Test"
- PricePerKWh = 0.25
- StartHour = 00:00
- EndHour = 23:59

Datos esperados:

Tarifa insertada con Id > 0.

Datos obtenidos:

Id generado correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Insertar tarifa null

Objetivo:

Comprobar que el sistema controla correctamente la inserción de valores nulos.

Datos de entrada:

null

Datos esperados:

Excepción `ArgumentNullException`.

Datos obtenidos:

Excepción lanzada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Actualizar tarifa existente

Objetivo:

Verificar que una tarifa existente puede ser modificada correctamente.

Datos de entrada:

Tarifa previamente insertada con modificación en PricePerKWh.

Datos esperados:

Resultado true.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Actualizar tarifa inexistente

Objetivo:

Comprobar que el sistema no actualiza registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar tarifa existente

Objetivo:

Comprobar que una tarifa puede eliminarse correctamente.

Datos de entrada:

Id válido previamente insertado.

Datos esperados:

Resultado true.

Datos obtenidos:

Eliminación correcta.

Resultado: ✓ Correcto

◆ Prueba 7 – Borrar tarifa inexistente

Objetivo:

Verificar que el sistema controla correctamente la eliminación de registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

SERVICE USER

◆ Prueba 1 – Listar usuarios

Objetivo:

Comprobar que el sistema devuelve correctamente la lista de usuarios almacenados en la base de datos junto con sus sesiones de carga asociadas.

Datos de entrada:

Sin parámetros.

Datos esperados:

Lista no nula de usuarios.

Datos obtenidos:

Lista correctamente devuelta.

Resultado: ✓ Correcto

◆ Prueba 2 – Insertar usuario válido

Objetivo:

Verificar que se inserta correctamente un nuevo usuario en la base de datos.

Datos de entrada:

User válido:

- FullName = "Usuario Test"
- Email = "test@gmail.com"
- RFIDTag = cadena única válida (≤ 20 caracteres)
- Balance = 10.00

Datos esperados:

Usuario insertado con Id > 0.

Datos obtenidos:

Id generado correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Insertar usuario null

Objetivo:

Comprobar que el sistema controla correctamente la inserción de valores nulos.

Datos de entrada:

null

Datos esperados:

Excepción ArgumentNullException.

Datos obtenidos:

Excepción lanzada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Actualizar usuario existente

Objetivo:

Verificar que un usuario existente puede ser modificado correctamente.

Datos de entrada:

Usuario previamente insertado con modificación en FullName o Balance.

Datos esperados:

Resultado true.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Actualizar usuario inexistente

Objetivo:

Comprobar que el sistema no actualiza registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar usuario existente

Objetivo:

Comprobar que un usuario puede eliminarse correctamente.

Datos de entrada:

Id válido previamente insertado.

Datos esperados:

Resultado true.

Datos obtenidos:

Eliminación correcta.

Resultado: ✓ Correcto

◆ Prueba 7 – Borrar usuario inexistente

Objetivo:

Verificar que el sistema controla correctamente la eliminación de registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

SERVICE CHARGING SESSION

◆ Prueba 1 – Listar sesiones de carga

Objetivo:

Comprobar que el sistema devuelve correctamente la lista de sesiones de carga almacenadas en la base de datos junto con el usuario y el cargador asociados.

Datos de entrada:

Sin parámetros.

Datos esperados:

Lista no nula de sesiones.

Datos obtenidos:

Lista correctamente devuelta.

Resultado: ✓ Correcto

◆ Prueba 2 – Insertar sesión válida

Objetivo:

Verificar que se inserta correctamente una nueva sesión de carga asociada a un usuario y a un cargador existentes.

Datos de entrada:

ChargingSession válida:

- ChargerId válido
- UserId válido
- StartTime = valor TimeSpan válido
- EndTime = null o valor válido
- KWhConsumed = valor decimal válido
- TotalCost = valor decimal válido

Datos esperados:

Sesión insertada con Id > 0.

Datos obtenidos:

Id generado correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Insertar sesión null

Objetivo:

Comprobar que el sistema controla correctamente la inserción de valores nulos.

Datos de entrada:

null

Datos esperados:

Excepción ArgumentNullException.

Datos obtenidos:

Excepción lanzada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Actualizar sesión existente

Objetivo:

Verificar que una sesión existente puede ser modificada correctamente.

Datos de entrada:

Sesión previamente insertada con modificación en:

- EndTime
- KWhConsumed
- TotalCost

Datos esperados:

Resultado true.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Actualizar sesión inexistente

Objetivo:

Comprobar que el sistema no actualiza registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar sesión existente

Objetivo:

Comprobar que una sesión puede eliminarse correctamente.

Datos de entrada:

Id válido previamente insertado.

Datos esperados:

Resultado true.

Datos obtenidos:

Eliminación correcta.

Resultado: ✓ Correcto

◆ Prueba 7 – Borrar sesión inexistente

Objetivo:

Verificar que el sistema controla correctamente la eliminación de registros inexistentes.

Datos de entrada:

Id = -9999

Datos esperados:

Resultado false.

Datos obtenidos:

Devuelve false.

Resultado: ✓ Correcto

5. Pruebas realizadas sobre ViewModels

MAIN WINDOW VIEWMODEL

◆ Prueba 1 – Estado inicial

Objetivo:

Comprobar que al instanciar el ViewModel se carga por defecto la vista principal.

Datos de entrada:

Instanciación del MainWindowViewModel.

Datos esperados:

CurrentView contiene una instancia de PrincipalViewModel.

Datos obtenidos:

CurrentView correctamente inicializado.

Resultado: ✓ Correcto

◆ Prueba 2 – Navegación a vista principal

Objetivo:

Verificar que el comando ShowMainCommand cambia correctamente la vista actual.

Datos de entrada:

Ejecución de ShowMainCommand.

Datos esperados:

CurrentView = PrincipalViewModel.

Datos obtenidos:

Vista cambiada correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Navegación a estaciones

Objetivo:

Comprobar que ShowStationsCommand cambia la vista actual.

Datos de entrada:

Ejecución de ShowStationsCommand.

Datos esperados:

CurrentView = StationsViewModel.

Datos obtenidos:

Vista cambiada correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Navegación a estaciones CRUD

Objetivo:

Comprobar que ShowStationsCRUDCommand cambia la vista actual.

Datos de entrada:

Ejecución de ShowStationsCRUDCommand.

Datos esperados:

CurrentView = StationsCrudViewModel.

Datos obtenidos:

Vista cambiada correctamente.

Resultado: ✓ Correcto

STATIONS VIEWMODEL

◆ Prueba 1 – Estado inicial

Objetivo:

Verificar que la colección de estaciones visibles se inicializa correctamente.

Datos de entrada:

Instanciación del ViewModel.

Datos esperados:

ListaEstacionesVisibles no es null y está vacía.

Datos obtenidos:

Colección inicializada correctamente.

Resultado: ✓ Correcto

◆ Prueba 2 – Cargar estaciones

Objetivo:

Comprobar que al ejecutar CargarCommand se cargan las estaciones desde la base de datos.

Datos de entrada:

Ejecución de CargarCommand.

Datos esperados:

ListaEstacionesVisibles contiene los registros de la base de datos.

Datos obtenidos:

Lista cargada correctamente.

Resultado: ✓ Correcto

STATIONS CRUD VIEWMODEL

◆ Prueba 1 – Nueva estación

Objetivo:

Comprobar que al ejecutar NuevaEstacionCommand se inicializan correctamente los valores.

Datos de entrada:

Ejecución de NuevaEstacionCommand.

Datos esperados:

- Id = 0
- IsActive = true
- 1 cargador por defecto

Datos obtenidos:

Valores inicializados correctamente.

Resultado: ✓ Correcto

◆ Prueba 2 – Nuevo cargador

Objetivo:

Verificar que al ejecutar NuevoCargadorCommand se añade un nuevo cargador a la colección.

Datos de entrada:

Ejecución de NuevoCargadorCommand.

Datos esperados:

Chargers.Count aumenta en 1.

Datos obtenidos:

Cargador añadido correctamente.

Resultado: ✓ Correcto

◆ Prueba 3 – Borrar cargador

Objetivo:

Comprobar que se elimina correctamente un cargador cuando hay más de uno.

Datos de entrada:

Ejecución de BorrarCargadorCommand con un cargador válido.

Datos esperados:

Chargers.Count disminuye en 1.

Datos obtenidos:

Cargador eliminado correctamente.

Resultado: ✓ Correcto

◆ Prueba 4 – Guardar estación (Insertar)

Objetivo:

Verificar que se inserta correctamente una nueva estación con sus cargadores.

Datos de entrada:

Estación nueva con datos válidos.

Datos esperados:

Id > 0 tras guardar.

Datos obtenidos:

Estación insertada correctamente.

Resultado: ✓ Correcto

◆ Prueba 5 – Guardar estación (Actualizar)

Objetivo:

Comprobar que una estación existente puede modificarse correctamente.

Datos de entrada:

Estación previamente creada con modificación en Name.

Datos esperados:

Cambio persistido en la base de datos.

Datos obtenidos:

Actualización realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 6 – Borrar estación

Objetivo:

Verificar que se eliminan correctamente la estación y sus cargadores asociados.

Datos de entrada:

Ejecución de BorrarEstacionCommand.

Datos esperados:

Estación eliminada de la base de datos.

Datos obtenidos:

Eliminación realizada correctamente.

Resultado: ✓ Correcto

◆ Prueba 7 – SelectedItemChanged

Objetivo:

Comprobar que al seleccionar una estación se cargan correctamente sus datos en el formulario.

Datos de entrada:

Objeto Station válido.

Datos esperados:

Propiedades del ViewModel actualizadas con los datos del objeto seleccionado.

Datos obtenidos:

Datos sincronizados correctamente.

Resultado: ✓ Correcto

PRINCIPAL VIEWMODEL

◆ Prueba 1 – Instanciación

Objetivo:

Comprobar que el ViewModel puede instanciarse correctamente.

Datos de entrada:

Instanciación del PrincipalViewModel.

Datos esperados:

Objeto creado sin excepciones.

Datos obtenidos:

Instanciación correcta.

Resultado: ✓ Correcto

6. Conclusiones

Las pruebas realizadas han permitido verificar el correcto funcionamiento de las operaciones principales del sistema, detectando posibles errores y validando el comportamiento esperado en escenarios normales y excepcionales.